UNIVASSOURAS
CAMPUS UNIVERSITÁRIO DE SAQUAREMA
CURSO DE ENGENHARIA DE SOFTWARE

MODELAGEM DE REDE DE ROTAS EM NEO4J (NOSQL/GRAFOS)

Saquarema
2026

Alfredo Menezes, 200, Bacaxá Saquarema/ RJ | CEP.: 28.994-675
universidadedevassouras.edu.br

MODELAGEM DE REDE DE ROTAS EM NEO4J (NOSQL/GRAFOS)

AUTOR GABRIEL PEREIRA MARINS MOREIRA - RA: 202422971

Trabalho de pesquisa e construção de conhecimento apresentado como requisito parcial para obtenção da nota avaliativa semestral 2026.2 da disciplina Banco de Dados não Relacional do curso Engenharia de Software na Univassouras – Campus Universitário de Saquarema.
Prof. Prof. Mesc. Diego Ramos Inácio

Saquarema
2026

Sumário

Repositório e entrega..... 4
Objetivo e arquitetura..... 4
Execução do projeto..... 5
Evidências dos testes..... 5
Testes no Neo4j..... 6
Declaração de uso de inteligência artificial..... 6
Autoavaliação de entendimento e domínio..... 7

Alfredo Menezes, 200, Bacaxá Saquarema/ RJ | CEP.: 28.994-675
universidadedevassouras.edu.br

Repositório e entrega

URL do repositório GitHub https://github.com/Moreira-2203/livraria_api/tree/master

Branch e/ou hash do commit enviado “Master”

Objetivo e arquitetura

O projeto disponibiliza dados de uma livraria migrados para o Neo4j por meio de uma API REST construída com Python e FastAPI. As operações acessam diretamente o banco de grafos usando o driver oficial neo4j e consultas Cypher, sem ORM relacional.

Tecnologias utilizadas

Componente	Utilização
Python	Linguagem do backend
FastAPI / Uvicorn	Rotas HTTP e servidor de desenvolvimento
Neo4j / Cypher	Banco de grafos, consultas e relacionamentos
Swagger UI	Execução manual das rotas e registro de evidências
GitHub	Versionamento e disponibilização do código-fonte

Entidades e relacionamentos

Nós principais: Autor, Livro, Categoria, Cliente e Pedido. Relações: ESCRITO_POR, PERTENCE_A, FEITO_POR e CONTEM; esta última armazena quantidade e preço unitário dos itens.

Rotas implementadas

Método	Endpoint	Finalidade
GET	/livros	Listar livros
GET	/livros/{id}	Detalhar livro, autor e categoria
POST	/clientes	Cadastrar cliente
POST	/pedidos	Criar pedido e relacionar itens
GET	/relatorios/mais-vendidos	Ranking agregado de vendas
GET	/livros/busca	Buscar livros pelo título (rota extra)

Execução do projeto

Pré-requisitos: Python, dependências do requirements.txt e instância Neo4j em execução. Configurar um arquivo .env local conforme .env.example, sem publicar credenciais.

Comando de instalação: python -m pip install -r requirements.txt

Comando de inicialização: python -m uvicorn main:app --reload

Swagger: http://127.0.0.1:8000/docs

Observações sobre configuração / ambiente de teste

O projeto foi desenvolvido e testado utilizando Python com o framework FastAPI.

O banco de dados utilizado foi o Neo4j, executado localmente na porta 7687, contendo os dados da livraria migrados anteriormente do PostgreSQL.

A conexão entre a aplicação e o Neo4j foi realizada por meio do driver oficial neo4j para Python, com as credenciais configuradas em um arquivo .env.

O servidor da API foi iniciado utilizando Uvicorn, disponibilizando as rotas em http://127.0.0.1:8000.

Os testes foram realizados pela interface Swagger UI (/docs) e as operações de leitura, escrita e relacionamento foram verificadas diretamente no Neo4j Browser.

Todas as seis rotas implementadas foram testadas com sucesso. Também foram realizados testes de tratamento de erros, incluindo livro inexistente, tentativa de cadastro com e-mail duplicado e pedido contendo livro não cadastrado.

Evidências dos testes

Os resultados foram baseados nos testes executados durante o desenvolvimento.

(as imagens estão no .zip anexado ao envio para melhor visualização)

GET /livros

GET /livros/1

POST /clientes

POST /pedidos

GET /relatorios/mais-vendidos

GET /livros/busca?titulo=Segredo

GET /livros/999999999

ST /clientes (e-mail repetido)

POST /pedidos (livro 999999)

Métrica de testes

Foram observados 9 retornos HTTP esperados em 9 cenários manuais documentados (6 cenários de sucesso e 3 de erro). Métrica de conformidade dos cenários executados: 9/9 = 100%. Essa métrica não equivale à cobertura automatizada do código.

Persistência e integridade dos dados

O cadastro de teste gerou o cliente ID 1001; o pedido de teste gerou o pedido ID 2001 e total de R$ 320,16. A consulta de relacionamentos do Neo4j retornou os dois itens do pedido. A busca por “Segredo” retornou 39 livros no Neo4j; os primeiros resultados coincidiram com a API.

Testes no Neo4j

(as imagens estão no .zip anexado ao envio para melhor visualização)

Testes negativos

O POST /clientes com e-mail já existente retornou HTTP 409 e o Neo4j apresentou somente um cliente com o e-mail testado. O POST /pedidos com livro inexistente retornou HTTP 404, sem relacionamento CONTEM para o livro 999999. O GET /livros/999999999 retornou HTTP 404.

Declaração de uso de inteligência artificial

Ferramenta utilizada: ChatGPT e Claude

Finalidade: apoio didático na estruturação da API, exemplos de FastAPI e Cypher, correção de conexão Neo4j/Python, orientação de execução e planejamento dos testes.

Partes apoiadas: estrutura do backend, consultas Cypher, tratamento de erros, casos de teste e organização da documentação. O código foi executado localmente e os resultados foram conferidos por meio do Swagger e do Neo4j Browser.

Métrica quantitativa: 9 de 9 cenários manuais documentados apresentaram os códigos HTTP esperados; 6 rotas da API testadas com sucesso.

Justificativa qualitativa: a IA auxiliou na aprendizagem do fluxo requisição HTTP - FastAPI - Cypher - Neo4j - JSON. Os testes práticos permitiram verificar os dados reais, a navegação de relacionamentos e o tratamento de entradas inválidas.

Autoavaliação de entendimento e domínio

Rota	Nota (1–5)	Justificativa pessoal
GET /livros	4	Compreendi bem o funcionamento da rota e sua finalidade de consultar os livros armazenados no Neo4j. Consegui executar os testes e interpretar o retorno JSON. Ainda preciso aprofundar meus conhecimentos sobre otimização de consultas e organização do código.
GET /livros/{id}	4	Entendi como a API consulta um livro específico e utiliza os relacionamentos do grafo para recuperar informações do autor e da categoria. Consegui verificar os resultados no Swagger e compreender o tratamento de erros para IDs inexistentes.
POST /clientes	4	Compreendi o processo de cadastro de clientes e a utilização de requisições POST para inserir informações no banco. Também consegui verificar a persistência dos dados e o tratamento de e-mails duplicados. Ainda tenho pontos a desenvolver sobre validações e segurança.
POST /pedidos	3	Compreendi a lógica geral da criação de pedidos e dos relacionamentos entre clientes, pedidos e livros. Entretanto, por envolver múltiplas operações e propriedades nos relacionamentos, ainda tenho dificuldades para compreender completamente as transações e implementar essa funcionalidade sem auxílio.
GET /relatorios/mais-vendidos	3	Entendi o objetivo da rota e como as agregações Cypher são utilizadas para calcular a quantidade de livros vendidos e o faturamento. Consegui executar e comparar os resultados, mas ainda preciso praticar a construção de consultas com funções de agregação e agrupamento de dados.